

NumPy Complete Guide (Keith Galli video — simplified)

Based on: Complete NumPy Data Science Tutorial Breakdown

Context: NumPy arrays vs standard Python lists configuration

Socho NumPy list ka "turbocharged" version hai. Python ki normal list slow hai bade data pe kaam karte waqt kyunki har element alag jagah memory mein store hota hai. NumPy array (**ndarray**) saara data ek continuous block mein rakhta hai — isliye math operations bahut fast hote hain, aur tu poore array pe ek line mein operation kar sakta hai (loop likhne ki zarurat nahi).

```
import numpy as np
```

Bas yehi ek line se shuru hota hai sab kuch.

1. The Basics — Arrays banana aur unke properties

```
a = np.array([1, 2, 3])          # 1D array – isse "vector" bolte hain
b = np.array([[9,8,7],[6,5,4]]) # 2D array – isse "matrix" bolte hain
```

Ek array ke 3 important properties hote hain, jaise ek object ke attributes hote hain OOP mein:

- **a.ndim** → kitne dimensions hain (1D, 2D, 3D...)
- **a.shape** → (**rows, columns**) — batata hai array ka "shape" kaisa hai
- **a.dtype** → array ke andar ke numbers kis type ke hain (int32, float64, etc.)

```
a.ndim    # 1
b.shape    # (2, 3) -> 2 rows, 3 columns
a.dtype    # dtype('int64')
```

Size aur itemsize:

- **a.itemsize** → ek element kitni memory leta hai (bytes mein)
- **a.size** → total kitne elements hain
- **a.nbytes** → total memory (**itemsize × size**)

Yeh isliye important hai kyunki NumPy ka poora fayda hi memory-efficient hone mein hai — isse pata chalta hai actually kitni memory bach rahi hai list ke comparison mein.

2. Accessing/Changing elements, rows, columns

Yeh part almost Python list jaisa hi hai, bas 2D/3D mein thoda different syntax.

```
a = np.array([[1,2,3,4,5,6,7],
              [8,9,10,11,12,13,14]])

a[1, 5]          # row 1, column 5 -> 13
a[0, :]          # pura row 0
a[:, 2]          # pura column 2 (dono rows ka 3rd element)
a[0, 1:6:2]      # row 0 mein, index 1 se 6 tak, 2 ke step mein (slicing)
```

Element change karna bilkul simple hai:

```
a[1,5] = 20      # single element change
a[:,2] = [1,2]   # pura column replace
```

3D arrays thoda mushroom hote hain samajhne mein, lekin sochne ka trick yeh hai: outer se inner jaao.

```
b = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
b[0,1,1]      # -> 4 (0th block, 1st row, 1st element)
```

Bas ek layer at a time peel karo, jaise nested lists.

3. Initializing Different Types of Arrays

Ye "shortcut functions" hain jab tumhe khud khud numbers nahi likhne, sirf shape chahiye:

```
np.zeros((2,3))      # sab zero ka 2x3 array
np.ones((4,2,2))     # sab ek ka array
np.full((2,2), 99)   # ek fix number se bhara array
np.full_like(a, 4)   # 'a' jaisa shape, par sab 4 se bhara
np.random.rand(4,2)  # random decimal numbers (0 to 1)
np.random.randint(-4,8, size=(3,3)) # random integers ek range mein
np.identity(5)       # identity matrix (linear algebra wala)
np.repeat(arr, 3, axis=0) # array ke elements repeat karna
```

Practice challenge jo video mein aata hai: ek 5x5 matrix banana jisme border pe 1 ho aur beech mein 9 aur baaki 0. Iska trick: pehle `np.ones((5,5))` banao, phir beech mein `np.zeros((3,3))` daal do, phir center element ko 9 set karo. Yeh dikhata hai ki slicing se assign karna kitna powerful hai.

4. Copying Arrays — Sabse Bada Trap ⚠

Yeh video ka sabse important warning hai:

```
a = np.array([1,2,3])
b = a          # GALTII! Yeh sirf reference copy hai
b[0] = 100
print(a)       # a bhi change ho jayega! [100, 2, 3]
```

`b = a` karne se `b` aur `a` dono same memory location ko point karte hain — jaise do naam ek hi ghar ke. Agar tumhe actual alag copy chahiye:

```
b = a.copy()   # ab yeh independent hai
```

Isko ignore karna bahut common bug hai jab log NumPy naye naye seekhte hain — Python lists mein bhi yeh same behavior hota hai actually, but NumPy mein specially matter karta hai kyunki hum bade data pe kaam karte hain.

5. Mathematics

Har operation directly poore array pe element-by-element apply hota hai — koi loop nahi chahiye:

```
a = np.array([1,2,3,4])
a + 2        # [3,4,5,6]
a - 2
a * 2
a / 2
a ** 2       # square karna
np.sin(a)
np.cos(a)
```

Do arrays ko bhi add/multiply kar sakte ho (agar shape match kare):

```
a + b        # element-wise addition
```

Yeh cheez "vectorization" kehlaati hai — ismein hi NumPy ki asli speed hai.

6. Linear Algebra

```
a = np.ones((2,3))
b = np.full((3,2), 2)
np.matmul(a, b)    # matrix multiplication (rows x columns rule follow hota hai)

c = np.identity(3)
np.linalg.det(c)    # determinant nikalna
```

Yeh section un logon ke liye zyada relevant hai jo ML/stats mein deep jaana chahte hain — matrix multiply, determinant, inverse, eigenvalues jaisi cheezein ***np.linalg*** module mein milti hain.

7. Statistics

```
stats = np.array([[1,2,3],[4,5,6]])

np.min(stats)          # sabse chota number
np.max(stats, axis=1)   # har row ka max (axis=1 -> row-wise)
np.sum(stats, axis=0)   # column-wise sum (axis=0 -> column ke through)
```

Axis samajhne ka trick: ***axis=0*** matlab "neeche ki taraf jaao" (column ke through operate karo), ***axis=1*** matlab "side mein jaao" (row ke through operate karo). Yeh confusing lagta hai shuru mein, bas yaad rakho: axis 0 = rows collapse hoti hain, axis 1 = columns collapse hoti hain.

8. Reorganizing Arrays

```
before = np.array([[1,2,3,4],[5,6,7,8]])
after = before.reshape((8,1))          # shape change, total elements same rehna chahiye

v1 = np.array([1,2,3,4])
v2 = np.array([5,6,7,8])
np.vstack([v1,v2])    # vertically stack karna (rows ki tarah upar-neeche)
np.hstack([v1,v2])    # horizontally stack karna (side by side)
```

reshape mein sirf yeh dhyan rakhna hai ki original size aur naya shape ka total product match kare (e.g. 8 elements ko 2x4 ya 4x2 ya 8x1 mein reshape kar sakte ho, but 3x3 mein nahi).

9. Load Data from File

```
filedata = np.genfromtxt('data.txt', delimiter=',')
filedata = filedata.astype('int32')    # type convert karna
```

Yeh csv/text file se numbers directly array mein load karta hai.

10. Boolean Masking & Advanced Indexing

Yeh video ka sabse "cool" part hai — filtering ka superpower:

```

filedata > 50          # yeh True/False ka array deta hai, same shape
filedata[filedata > 50] # sirf woh values deta hai jo condition satisfy karti hain

np.any(filedata > 50, axis=0) # kya kisi column mein koi value 50 se badi hai?
np.all(filedata > 50, axis=0) # kya SAARI values 50 se badi hain?

((filedata > 50) & (filedata < 100)) # multiple conditions combine karna

```

Advanced indexing — list ke andar list daal ke multiple specific indices access kar sakte ho:

```

a = np.array([1,2,3,4,5,6,7,8,9])
a[[1,2,8]] # index 1, 2, aur 8 ke elements ek saath

```

Yeh boolean masking wala concept pandas mein bhi exactly waise hi use hota hai (filtering rows condition ke basis pe) — isliye yeh seekhna direct pandas ke liye foundation hai.

Quick Summary (cheat-sheet style)

Kaam	Function
Array banana	<code>np.array()</code>
Shape/dtype dekhna	<code>.shape</code> , <code>.dtype</code> , <code>.ndim</code>
Zero/one/random array	<code>np.zeros()</code> , <code>np.ones()</code> , <code>np.random.rand()</code>
Copy (safe)	<code>.copy()</code>
Math	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code> , <code>np.sin()</code>
Matrix multiply	<code>np.matmul()</code>
Stats	<code>np.min()</code> , <code>np.max()</code> , <code>np.sum(axis=)</code>
Reshape	<code>.reshape()</code>
Stack	<code>np.vstack()</code> , <code>np.hstack()</code>
Filter	<code>arr[arr > x]</code>

Ek line mein poora video: NumPy array = fast, memory-efficient list jisme tu direct math kar sakta hai bina loop likhe, aur jisme boolean conditions se filter bhi laga sakta hai — yehi cheez ise ML/data ke kaam ke liye backbone banati hai (pandas isi pe bana hai).